크로스 플랫폼 프로그래밍



9주: Context API

강의: 정종수



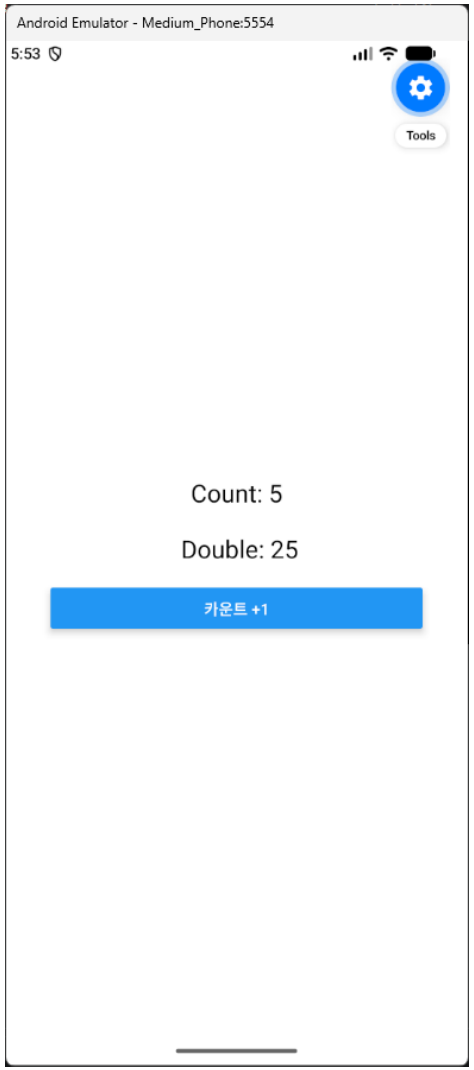
강의 내용

1. 상태 관리
2. 전역 상태 관리
3. Context API
4. createContext
5. useContext
6. 실습

▪ 상태 관리

상태 관리 종류	예	대표 관리방법/도구
지역 상태(Local State)	**입력 값, 버튼 상태, 모달 열림/닫힘**	**useState**
전역 상태 (Global State)	**로그인 사용자, 테마, 장바구니**	**Context API, Zustand, Redux Toolkit, Jotai**
서버 상태(Server State)	API에서 받은 게시글, 상품 목록	fetch, TanStack Query
영구 상태(Persistent State)	로그인 유지, 설정 값	AsyncStorage, SecureStore, MMKV
폼 상태(Form State)	회원가입, 로그인 입력 값	React Hook Form
복잡한 상태 흐름	로그인 단계, 결제 단계	useReducer, Redux

1. 지역 상태 관리



앱에서 바뀌는 값을 저장하고, 그 값이 바뀌면 화면도 같이 바뀌게 하는 것을 상태 관리라 한다.

- 가장 기본적인 상태관리는 useState는 지역 상태 관리이다.

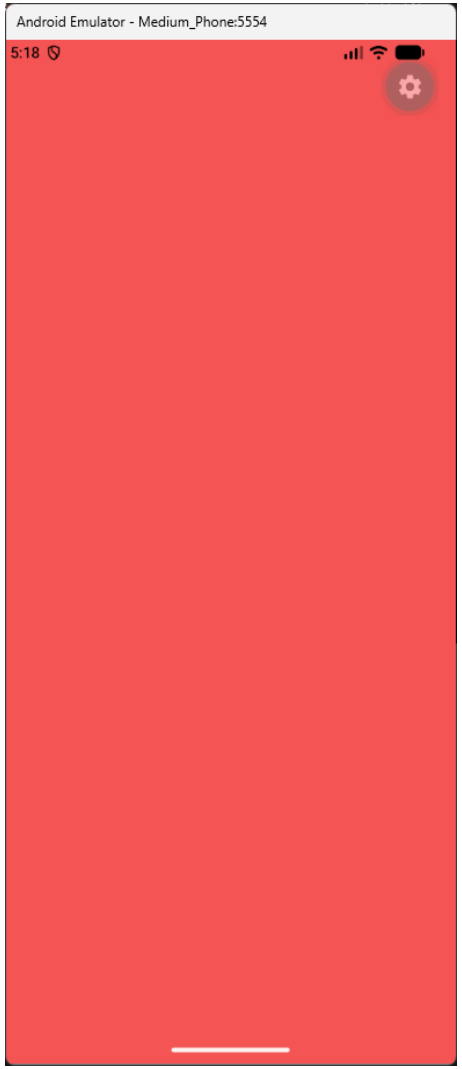
```
const [count, setCount] = useState(5);
```

현재 상태 값

상태 값을 바꾸는 함수

처음 값(초기 값)

2. 전역 상태 관리



- **App.jp**

```
import App from "./src/App";

export default App;
```

- **src폴더 App.jp**

```
import React from 'react';
import styled from 'styled-components/native';

const Container = styled.View`
  flex: 1;
  background-color: #f55353;
  justify-content: center;
  align-items: center;
`;
const App = () => {
  return <Container />;
};
export default App;
```

2. 전역 상태 관리

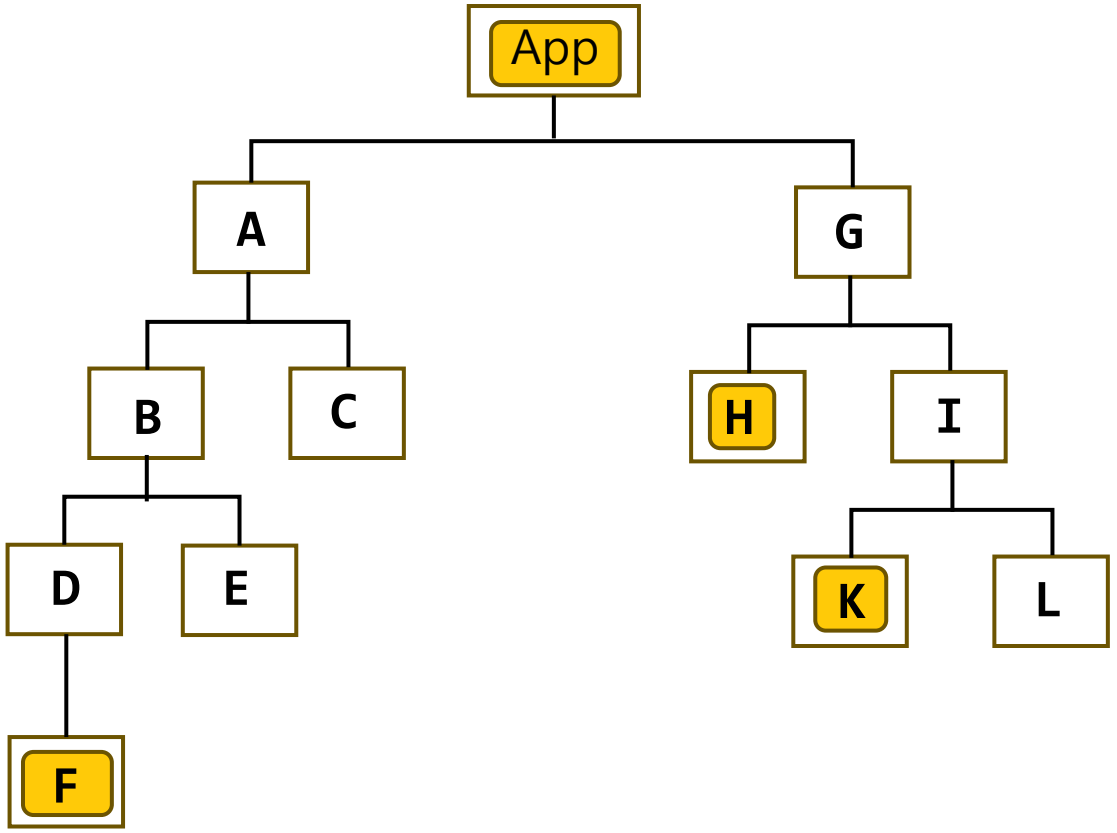


그림 7-1 일반적인 흐름

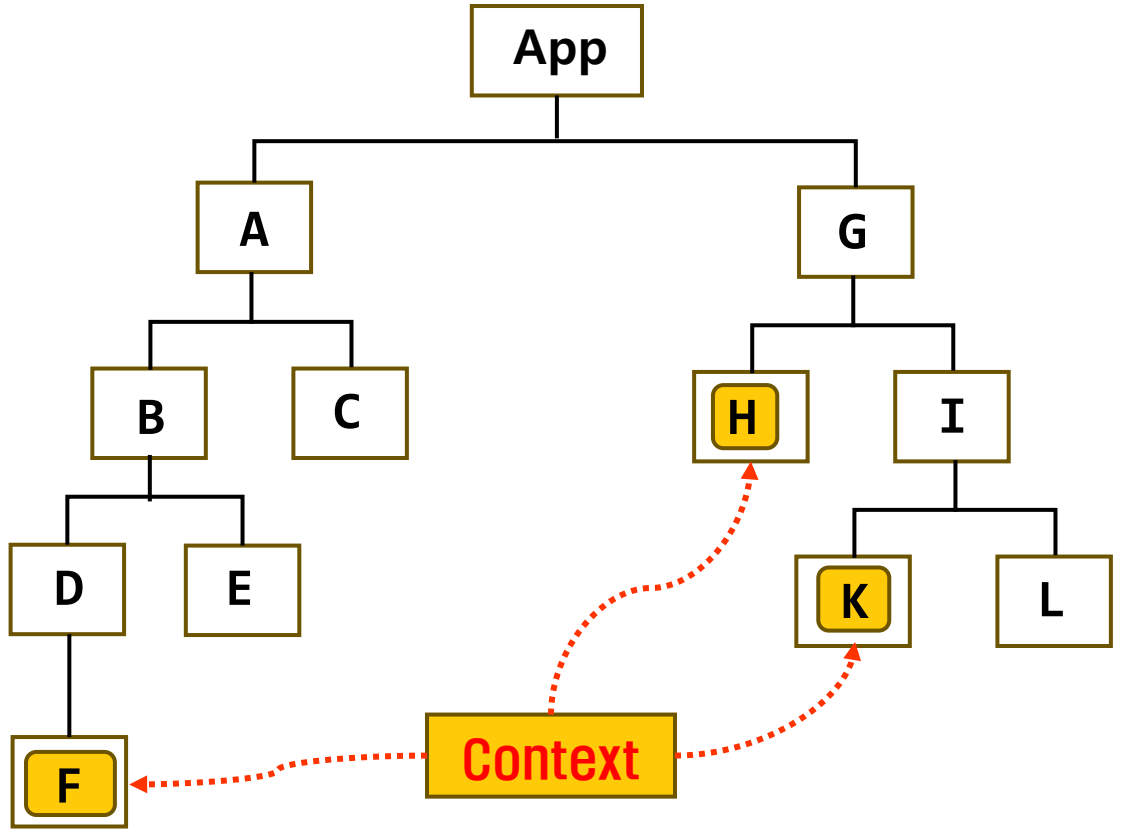


그림 7-2 Context API 흐름

2. 전역 상태 관리

React Native에서 전역 상태 관리란 앱 전체에서 여러 컴포넌트들이 공통으로 접근, 수정할 수 있는 상태(state)를 중앙에서 관리하는 방법.

즉, 각 화면이나 컴포넌트 간에 데이터를 전달하기 위해 props drilling(상위→하위로 계속 전달)을 반복하지 않고, 공통 저장소(store)에 데이터를 저장해 두고 어디서든지 접근할 수 있게 하는 구조.

- **전역 상태 관리가 필요한 이유**

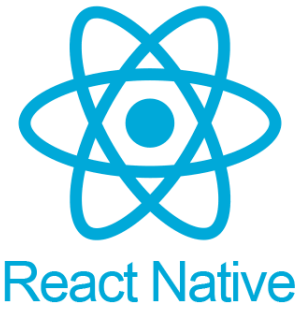
상황	문제점	전역 상태 관리의 효과
로그인 상태 관리	여러 화면에서 `user` 정보 필요	한 번 로그인하면 모든 화면에서 동일 정보 공유
다크모드 / 테마 설정	여러 컴포넌트에서 테마 색상 필요	하나의 전역 `theme` 상태로 전체 변경
장바구니 / 북마크	여러 화면에서 동일 데이터 접근	중앙 `store`에서 데이터 공유 및 업데이트
API 응답 캐싱	매번 서버 호출	전역 상태로 캐싱, 재활용

3. Context API

- **전역 상태 관리 Context API 핵심 3가지**

  - `createContext()`
    - Provider → 데이터를 제공
    - ~~Consumer → 데이터를 받아서 사용~~

  - `useContext()`



3. Context API: createContext 함수

createContext()는 React Native에서 전역 상태를 공유하기 위한 Context 객체를 생성하는 함수.

즉, 컴포넌트를 직접 props로 전달하지 않고도 하위 컴포넌트들이 같은 데이터를 공유하도록하는 전역 데이터 통로.

- 1. 기본 개념

```
const MyContext = createContext( defaultValue );
```

Provider와 Consumer를 포함한 객체

Provider가 없을 때 사용할 기본값

3. Context API: createContext 함수

- **2. Context의 핵심 구조**

구 분	역 할
createContext( )	Context 생성
Provider	데이터 제공
Consumer	데이터를 받아서 사용

- **3. 동작 흐름**

1) createContext( )로 Context 생성, Ex) const MyContext = createContext();
2) 상위 컴포넌트 Provider로 값 넣기, <MyContext.Provider value="Value">
3) 하위 컴포넌트 값 받기, const value = useContext(MyContext);
4) Provider의 value 값이 변경되면, 그 값을 사용하는 하위 컴포넌트의 화면도 자동으로 재렌더링.

4. createContext: Consumer

- 1. Consumer[데이터를 사용하는 쪽: Context의 “소비자”]
  - 역할: Provider가 제공한 값을 읽어 오는 컴포넌트
  - 문법: < UserContext.Consumer >

    {value => [. . .] }

    </UserContext.Consumer>.

- 2. 샘플

```
const User = () => {
  return (

    // UserContext.Provider에서 전달한 값을
    // Consumer를 통해 받아서 사용
    <UserContext.Consumer>
     {/* value에는 Context에서 전달된 데이터가 들어옴 예: { name: "홍길동" }   */}
     {(value) => (
       <Container>
         <StyledText> Name: {value.name} </StyledText>
       </Container>
     )}
    </UserContext.Consumer>
  );
};
```

Context Provider에서 공급된 name 변수 값 “홍길동”

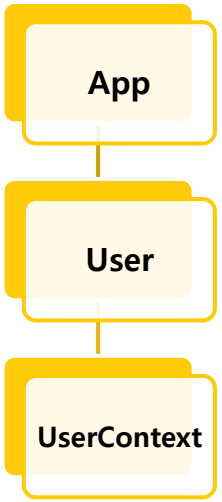
4. createContext: Consumer



- App.js

```
import React from "react";
import styled from "styled-components/native";
import User from "./src/components/User";


// 화면 전체를 감싸는 스타일 컴포넌트
const Container = styled.View`
  flex: 1;
  justify-content: center;
  align-items: center;
`;
      {/*
        value에는 Context에서 전달된 데이터가 들어옴
        예: { name: "홍길동" }
      */}
export default function App() {
  return (
      <Container>
        {/* User 컴포넌트 출력 */}
        <User />
      </Container>
  );
}
```



- Styled-components 패키지 설치
  npm install styled-components

4. createContext: Consumer



- **src/components/User.js**

```
import React, { useContext } from "react";
import styled from "styled-components/native";
import UserContext  from "../contexts/UserContext";
// 텍스트 스타일
const StyledText = styled.Text`
  font-size: 40px;
  color: blue;
`;
// 화면 전체 중앙 정렬
const Container = styled.View`
  flex: 1;
  justify-content: center;
  align-items: center;
`;
const User = () => {
  return (
    // UserContext.Provider에서 전달한 값을
    // Consumer를 통해 받아서 사용
    <UserContext.Consumer>
      {/* value에는 Context에서 전달된 데이터가 들어옴 예: { name: "홍길동" }   */}
      {(value) => (
        <Container>
          <StyledText> Name: {value.name} </StyledText>
        </Container>
      )}
    </UserContext.Consumer>
  );
};
export default User;
```

4. createContext: Consumer



- **src/contexts/UserContext.js**

```
import { createContext } from "react";

const UserContext = createContext({name: '홍길동'});

export default UserContext;
```

4. createContext: Provider

- 1. Provider[데이터를 공급하는 쪽: Context의 “제공자”]
  - 역할: Context에 저장된 값을 하위 트리에 공급하는 컴포넌트
  - 문법: < UserContext.Provider value = {Value} >
  - </UserContext.Provider>
  - Value로 전달한 값은 Provider 내부의 모든 컴포넌트서 접근할 수 있음.

- 2. 샘플

```
export default function App() {
  return (

    // Provider가 하위 컴포넌트에 사용자 정보를 제공
    <UserContext.Provider
      value={{
        name: "정종수",
        age: 25
      }}
    >
      <Container>
        <User />
      </Container>
    </UserContext.Provider>
  );
}
```

“정종수”라는 변수 값이 Context에 공급되어 (Provider) 하위 트리 전체에서 읽을 수 있음

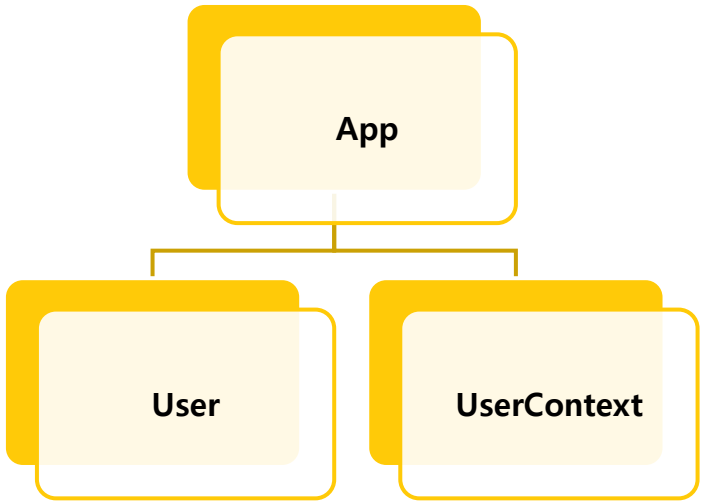
4. createContext: Provider



- App.js

```
import React from "react";
import styled from "styled-components/native";
import User from "./src/components/User";
import UserContext from "./src/contexts/UserContext";

const Container = styled.View`
  flex: 1;
  justify-content: center;
  align-items: center;
`;
export default function App() {
  return (
    // Provider가 하위 컴포넌트에 사용자 정보를 제공
    <UserContext.Provider
      value={{
        name: "정종수",
        age: 25
      }}
    >
      <Container>
        <User />
      </Container>
    </UserContext.Provider>
  );
}
```



4. createContext: Provider



- **src/components/User.js**

```
import React, { useContext } from "react";
import styled from "styled-components/native";
import UserContext  from "../contexts/UserContext";
// 텍스트 스타일
const StyledText = styled.Text`
  font-size: 40px;
  color: blue;
`;
// 화면 전체 중앙 정렬
const Container = styled.View`
  flex: 1;
  justify-content: center;
  align-items: center;
`;
const User = () => {
  return (
    // UserContext.Provider에서 전달한 값을
    // Consumer를 통해 받아서 사용
    <UserContext.Consumer>
      {/* value에는 Context에서 전달된 데이터가 들어옴 예: { name: "홍길동" }   */}
      {(value) => (
        <Container>
          <StyledText> Name: {value.name} </StyledText>
        </Container>
      )}
    </UserContext.Consumer>
  );
};
export default User;
```

4. createContext: Provider



- **src/contexts/UserContext.js**

```
import { createContext } from "react";

const UserContext = createContext({name: '홍길동'});

export default UserContext;
```

5. Context API: useContext 함수

useContext( )는 React Native에서 Context API를 사용할 때, 하위 컴포넌트가 Context 값(전역 상태)을 직접 꺼내서 사용하는 훅(Hook).

- 1. 기본 개념

```
const value = useContext( MyContext );
```

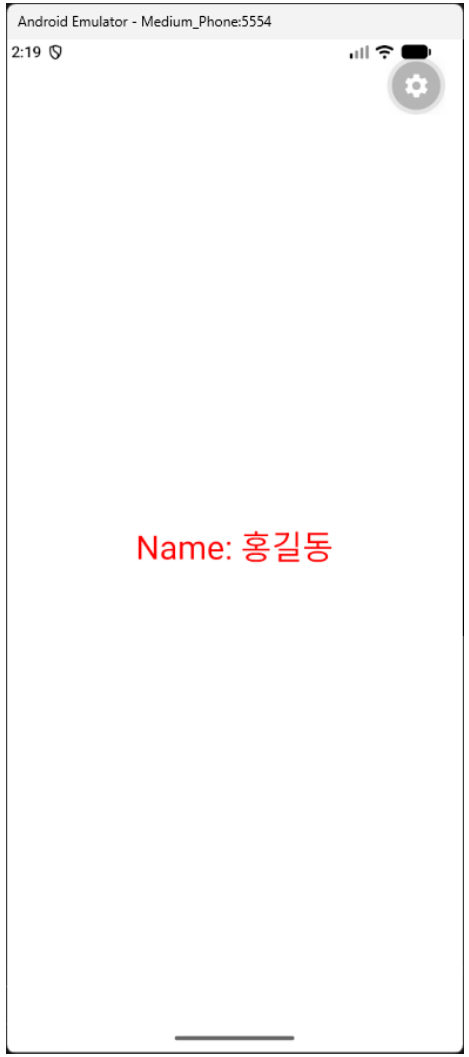
가장 가까운 <MyContext.Provider>가 제공하는 값

createContext( )로 만든 Context 객체

- 2. 동작 이해

  useContext(Context)는 React Native 내부에서 가장 가까운 Provider의 Value를 찾아 반환.

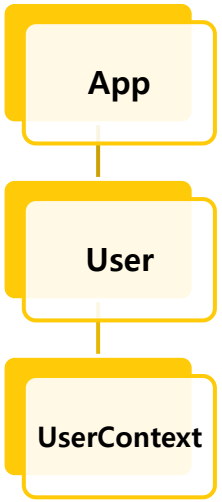
5. Context API: useContext 함수



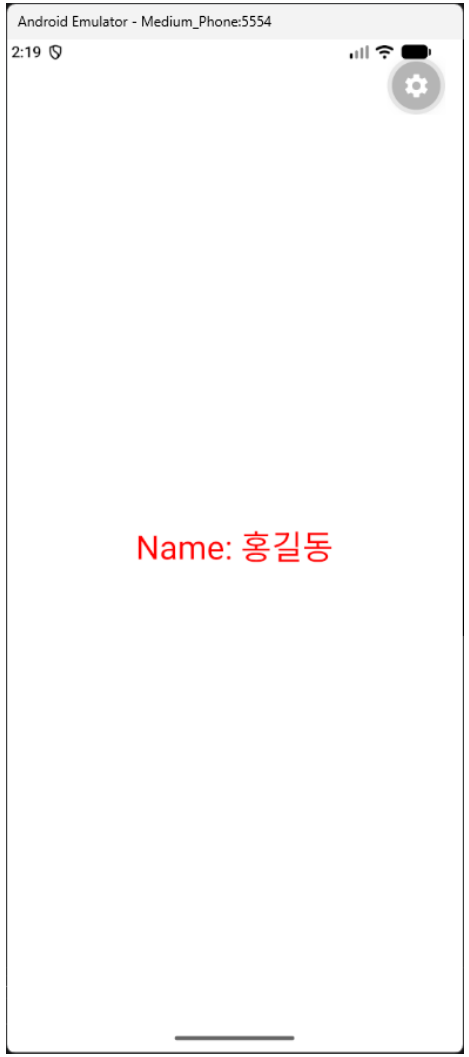
- App.js

```
import React from "react";
import styled from "styled-components/native";
import User from "./src/components/User";


// 화면 전체를 감싸는 스타일 컴포넌트
const Container = styled.View`
  flex: 1;
  justify-content: center;
  align-items: center;
`;
      {/*
        value에는 Context에서 전달된 데이터가 들어옴
        예: { name: "홍길동" }
      */}
export default function App() {
  return (
      <Container>
        {/* User 컴포넌트 출력 */}
        <User />
      </Container>
  );
}
```



5. Context API: useContext 함수



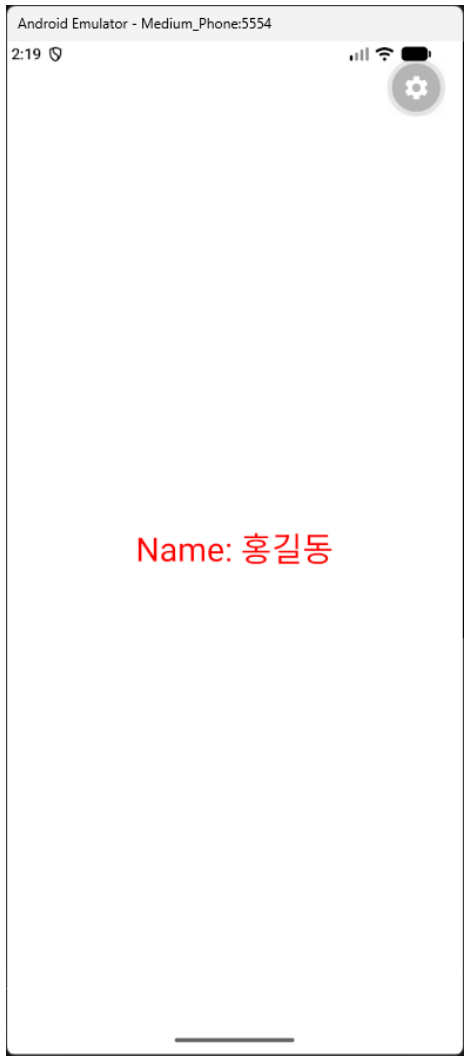
- **src/components/User.js**

```
import React, { useContext } from "react";
import styled from "styled-components/native";
import UserContext  from "../contexts/UserContext";

// 텍스트 스타일
const StyledText = styled.Text`
  font-size: 30px;
  color: red;
`;
// 화면 전체 중앙 정렬
const Container = styled.View`
  flex: 1;
  justify-content: center;
  align-items: center;
`;
const User = () => {
  const value = useContext(UserContext);


  return (
    <Container>
      <StyledText>
        Name: {value.name}
      </StyledText>
    </Container>
  );
};
export default User;
```

5. Context API: useContext 함수



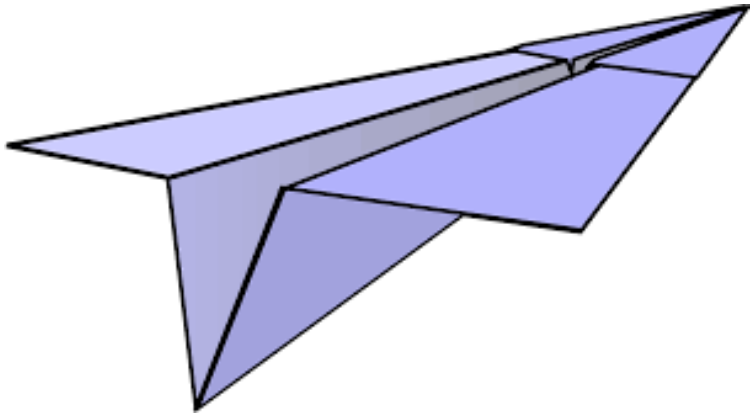
- **src/context/UserContext.js**

```
import { createContext } from "react";

const UserContext = createContext({name: '홍길동'});

export default UserContext;
```

수고 많으셨습니다.



Q&A

E-mail: jungjongsoo@yahoo.com